2025

TrueRepairs

CSC 4996 – SDS DOCUMENT

HISHAM ISSA, ROBERT SIMOVSKI, JIBRAN IQBAL, ISHMAEL GONZALES

Table of Contents

1. Introduction....3
   1.1 Purpose....3
   1.2 Audience....3
2. General Overview....3
   2.1 Design Philosophy....3
   2.2 Design Guidelines....3
      2.2.1 UI & Visual Design....3
      2.2.2 User Experience (UX)....4
      2.2.3 Responsiveness & Cross-Platform Support....4
      2.2.4 Backend & API Design Principles....4
      2.2.5 System Architecture & Security....4
   2.3 Design Assumptions....4
   2.4 Design Constraints....4
3. Architecture Design....5
   3.1 Hardware Architecture....5
   3.2 Software Architecture....6
   3.3 Security Architecture....7
      3.3.1 Access Control....7
      3.3.2 Data Protection....7
      3.3.3 Network Security....7
      3.3.4 Application & Physical Security....7
   3.4 Communication Architecture....8
      3.4.1 Client-Server Communication....8
      3.4.2 Backend External Services Communication....8
      3.4.3 Standard Request & Response Formats....8
   3.5 Performance Architecture....9
4. System Design....9
   4.1 Use Case....9

4.1.1 Use Case Diagram 9
4.2 Sequence Diagram 20
4.2.1 AI Scan Flow 20
4.2.2 User Login 21
4.2.3 AI View Final Report 22
4.2.4 User Registration 23
4.3 Data Flow Diagrams 24
4.3.1 TrueRepairs Data Flow Diagram - Level 0 24
4.3.2 TrueRepairs Data Flow Diagram - Level 1 (Frontend) 25
4.3.3 TrueRepairs Data Flow Diagram - Level 1 (Backend) 25
4.4 Class Diagram 26
4.5 API Endpoints Overview 26
4.6 User Interface Design 26
Figure 4.6.1 – Homepage 27
Figure 4.6.2 – Registration / Login Page 27
Figure 4.6.3 – User Dashboard 28
Figure 4.6.4 – Vehicle Information Dashboard 28
Figure 4.6.4 – AI Scan Page 29
Figure 4.6.5 – Repair Report Page 29
4.7 Database Design Diagram 30
5.0 References 31

1. Introduction

1.1 Purpose

The purpose of this Software Design Specification (SDS) is to define the architecture, design, and interaction logic of the TrueRepairs web application.
It provides a comprehensive view of the system's structure, including software modules, database schemas, communication flows, and security mechanisms.
This document bridges the gap between the Software Requirements Specification (SRS) and the actual implementation phase.

1.2 Audience

This document is intended for developers, project managers, and stakeholders involved in the development and maintenance of the TrueRepairs application.
It also serves as a reference for future developers working on extending or refactoring the system.

---

2. General Overview

2.1 Design Philosophy

TrueRepairs prioritizes functionality, accuracy, and transparency in vehicle repair cost estimation.
The system emphasizes modular design and a modern user experience, ensuring accessibility and performance on multiple devices.

2.2 Design Guidelines

2.2.1 UI & Visual Design

- Consistent color scheme (blue and white for clarity and trust).
- Clean, minimal interface with proper spacing.
- Standardized typography hierarchy for headings.
- Use of icons for better visuals affordance and clarity.
- Uniform styling for modals, popups, cards, and interactive components.

2.2.2 User Experience (UX)

- Core actions AI scan > Review Damages > Download Report must be reachable within three clicks.
- Progressive disclosure. Show advanced information only when needed.
- Clear loading information for AI processing, uploads, and report generation.
- Predictable actions for vehicle selection, image upload, and report generation.

2.2.3 Responsiveness & Cross-Platform Support

- Responsive design for iOS/Android with desktop scaling.
- Adaptive modals, carousels, and dropdowns for smaller screens sizes.
- Buttons and touch targets are sized for mobile users.
- High-performance rendering for large images and long report content.

2.2.4 Backend & API Design Principles

- Modular and well-structured Express routes
- Validation of all inputs to prevent data or unintended server behavior.
- Consistent JSON response structures with clear success.
- Maintainable code with comments for complex logic such as damage merging.

2.2.5 System Architecture & Security

- Layered system design with clear separation between frontend, backend, database, AI services, and image storage.
- Stateless APU using JWT authentication, with secure password handling.
- Scalable backend organization using controllers, routes, services, and middleware.

2.3 Design Assumptions

- Users will access TrueRepairs via a modern browser.
- External APIs (OpenAI, Cloudinary) remain available.
- Internet connectivity is required for AI-based scans and cloud interactions.

2.4 Design Constraints

- All data is processed via HTTPS using REST APIs.
- Heavy image processing occurs server-side to prevent frontend slowdown.
- System performance depends on third-party API response times.

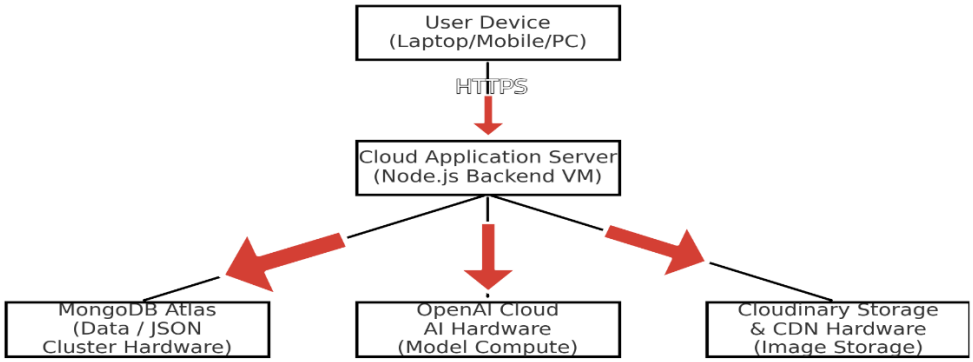
2.5 Design Standards

- Code formatting: **camelCase** (frontend) and **snake_case** (backend).
- Naming conventions follow consistency across modules.
- UI fonts use Sans-serif for readability and accessibility.
- Backend adheres to REST API design principles with JSON payloads.

---

3. Architecture Design

3.1 Hardware Architecture



TrueRepairs operates as a cloud-hosted web application accessible via web browsers. The backend runs on Node.js hosted by a cloud provider (Render/AWS), with MongoDB Atlas as the cloud database.

Hardware Components

- Users Devices: Desktop, laptop, tablet, or mobile devices with reliable internet connection.
- Cloud Applications Server: A cloud-hosted virtual machine or container instance running the backend services.

- Cloud Database Server: A managed MongoDB Altas cluster that stores users accounts, vehicle data, Manuel & AI reports.
- External AI: Cloud infrastructure operated by OpenAI.
- Cloud Media Storage: Cloudinary's globally distributed storage and CDN hardware and manages uploaded images.

3.2 Software Architecture

TrueRepairs follows a layered MERN (MongoDB, Express, React, Node.js) architecture.

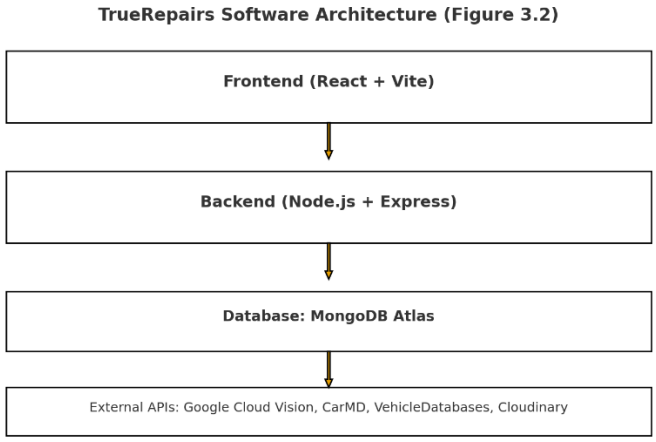
Layers:

1. **Frontend (React + Vite)** – Handles user interactions, forms, and visualizations.
2. **Backend (Node.js + Express)** – Processes requests, manages authentication, and integrates APIs.
3. **Database (MongoDB Atlas + Mongoose)** – Stores user accounts, vehicle data, and repair reports. Parts Pricing JSON Data (Local Application Resource) – Provides internal part names, pricing information, and labor-hour estimates used during cost estimation.
4. **External APIs** – OpenAI, Cloudinary.

Responsibilities:

- **Frontend:** User registration, login, AI scan upload, report display.
- **Backend:** Logic, authentication, API aggregation, and data management.
- **Database:** Structured collections with enforced schemas.
- **External Services:** AI vision detection, vehicle specs, and cost estimation.

Figure 3.2 – Software Architecture Diagram



Describes the layered communication between frontend, backend, database, and APIs.

3.3 Security Architecture

3.3.1 Access Control

- User authentication performed through JWT tokens, access for private endpoints.
- The password stored using bcrypt hashing with appropriate salting.
- Token expiration, renewal, and validation to prevent unauthorized access.
- Backend endpoints enforce strict authorization checks for protected actions.

3.3.2 Data Protection

- Cloudinary image URLs that store raw photo files.
- MongoDB collections enforce strict Mongoose schema validations.
- All incoming data is sanitized to prevent malicious data.
- Sensitive configuration items stored in server-side environments.

3.3.3 Network Security

- All client-server communications are encrypted.
- Backend network requests to external services.
- Strict CORS policy enforced to prevent unauthorized front-end origins.

3.3.4 Application & Physical Security

- Application logs exclude sensitive data.
- Regular package updates and dependencies patching.
- Failure isolation and multi-zone redundancy ensure continued operation even if individual hardware nodes fail.

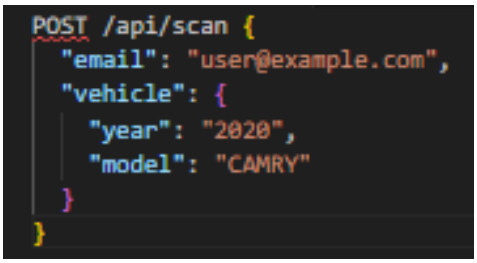
3.4 Communication Architecture

3.4.1 Client-Server Communication

Channel: HTTPS

Protocol: JSON API (Axios)

- The frontend sends JSON-encoded requests to backend endpoints for:
  - Authentication
  - AI Scan uploads
  - Manual vehicle data lookups
  - Report generation
  - User history and settings updates
- Responses use a standardized JSON schema containing status, message, and data fields.



3.4.2 Backend External Services Communication

1. OpenAI API (AI Image Analysis)

- Protocol: HTTPS
- Request Format: JSON or multipart form-data
- Response: JSON containing:
  - detected part labels
  - confidence scores
  - damage descriptions

2. Cloudinary (Image Upload & CDN)

- Protocol: HTTPS
- Request Format: multipart form-data

3. MongoDB Atlas (Database Cluster)

- Protocol: DNS-secured TLS-encrypted channel
- Format: BSON documents
- Operations: Queries, inserts, updates, deletes via Mongoose schema models

3.4.3 Standard Request & Response Formats

HTTP Status Codes Used

- 200 – Success
- 201 – Resource Created
- 400 – Bad Request
- 401 – Unauthorized
- 404 – Not Found
- 500 – Server Error

3.5 Performance Architecture

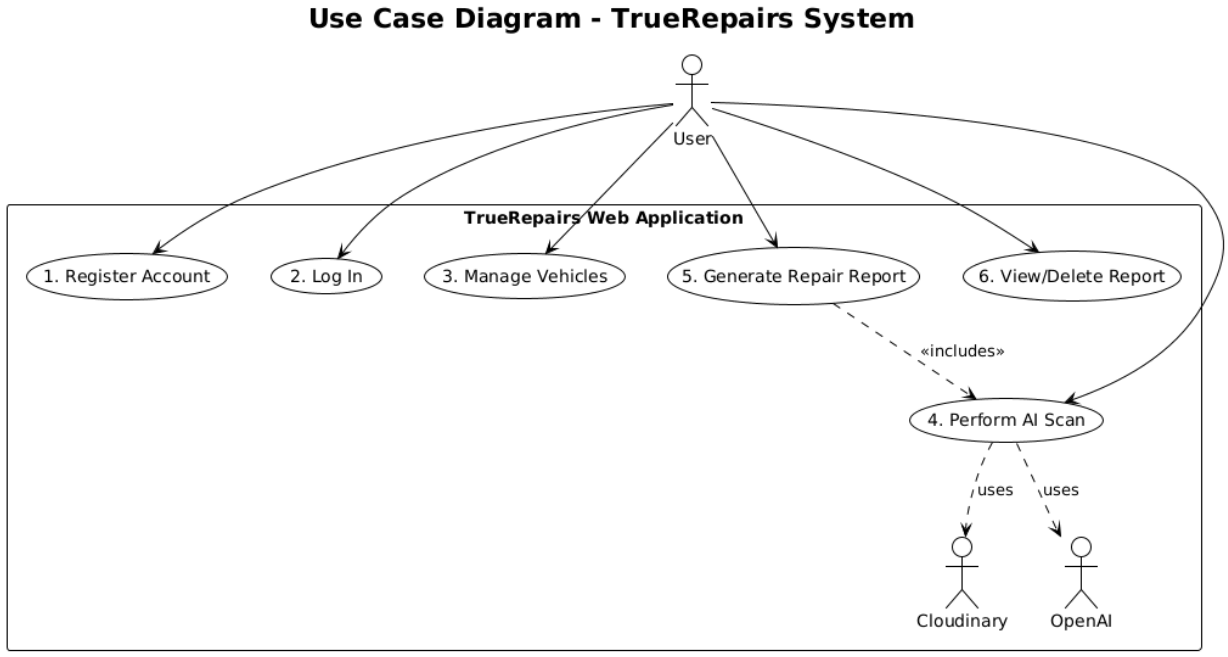
- Caching: In-memory caching for repeated vehicle queries.
- Indexes: Created on userId and vehicleId fields.
- Response Times: Standard requests under 500 ms; AI scan under 10 s.
- Async Operations: Non-critical tasks like email verification run asynchronously.

---

4. System Design

4.1 Use Case

4.1.1 Use Case Diagram



Description:
The diagram depicts how the User interacts with the TrueRepairs system.

It connects the actor to primary use cases such as registration, login, vehicle management, AI scanning, and report generation.
All external systems (OpenAI, Cloudinary) are treated as secondary actors connected through the backend.

Use Cases	External Systems
• Register Account • Log In • Manage Vehicles • Perform AI Scan • Generate Repair Report • View/Delete Report	• Cloudinary (API for image storage) • OpenAI (API for damage analysis)

Use Case ID: UC-01
Use Case Name: Register Account

Item	Description
Primary Actor	User
Goal in Context	Allow a new user to create an account on the TrueRepairs platform.
Scope	TrueRepairs Web Application
Level	User Goal
Preconditions	The user has Internet access and navigates to the registration page.
Trigger	The user clicks the “Sign Up” or “Get Started” button on the TrueRepairs Home page.
Postconditions	A verified user account is created and stored in MongoDB Atlas.
Main Flow	1. User opens Signup or Get Started button. 2. User enters name, email, and password. 3. System validates fields. 4. System hashes the password and stores data in MongoDB. 5. System displays success messages and redirects to login page.
Alternative Flow	A1. Invalid or duplicate email → system shows an error message and asks user to retry.
Exception Flow	E1. Server or network error → system displays “Registration Failed, Try Again Later.”

Use Case ID: UC-02
Use Case Name: User Login

Item	Description
Primary Actor	User
Goal in Context	Authenticate an existing user and provide secure access to the Dashboard.
Scope	TrueRepairs Web Application
Level	User Goal
Preconditions	A valid account already exists in the database.
Trigger	The user clicks “Login” button on the TrueRepairs home page.
Postconditions	A JSON Web Token (JWT) is issued, stored client-side, and used for session access.
Main Flow	1. Clicks Sign In button. 2. User enters credentials. 3. Backend verifies email and password. 4. System issues JWT and sends it to frontend. 5. Dashboard loads with personalized data.
Alternative Flow	A1. Incorrect credentials → display “Invalid Email or Password.”
Exception Flow	E1. Server timeout → prompt “Unable to connect, please retry.”

Use Case ID: UC-03
Use Case Name: Manage Vehicles

Item	Description
Primary Actor	User
Goal in Context	Allow users to add, edit, or remove vehicle information.
Scope	TrueRepairs Web Application
Level	User Goal
Preconditions	User is authenticated and on the Dashboard.
Trigger	User clicks the “My Vehicles” section on the Dashboard.
Postconditions	Vehicle records are created or updated in MongoDB Atlas.
Main Flow	**Add a New Vehicle** 1. User opens “My Vehicles.” System displays the vehicle list and buttons: “Add via VIN” and “Add Manually.” 2. User chooses one of the options: • VIN Lookup: User enters VIN → clicks “Search VIN.” • Manual Entry: User enters Make, Model, Year manually. 3. System validates the input. • Missing fields → red highlight • Invalid VIN → error message 4. System saves the new vehicle in MongoDB Atlas. 5. System refreshes the list showing the newly added vehicle. **View Vehicle Details** 6. User Click “View” on a specific vehicle **Download Report** 7. Inside the popup click “Download” **Delete Report** 8. User clicks “Delete” on vehicle entry. 9. User confirms deletion. 10. System removes the record from MongoDB

Item	Description
Alternative Flow	A1. VIN already exists → system notifies duplication. A2. User cancels delete → popup closes
Exception Flow	E1. Database error → show “Unable to save vehicle, try again.” E2. VIN API lookup fails → VIN lookup unavailable. Please try manual entry.

Use Case ID: UC-04
Use Case Name: Perform AI Scan

Item	Description
Primary Actor	User
Supporting Actors	None
Goal in Context	Allow a user to upload vehicle images and generate AI-based damage detection and cost estimation results.
Scope	TrueRepairs Web Application & Backend Server
Level	Sub-Function
Preconditions	User is logged in and has selected a vehicle to analyze.
Trigger	User clicks the “Upload Images” button for AI Scan in the AIScan popup.
Postconditions	Damage analysis and cost estimation results are stored temporarily as a pending report in MongoDB.
Main Flow	**A. User Uploads Images** 1. User opens the AI Scan popup. System displays an image upload interface. 2. User selects 1–6 vehicle images. Previews appear in the popup UI. 3. User enters their labor rate (or accepts default value). 4. User clicks “Run AI Scan.” System shows a loading indicator. **B. Image Processing** 5. System uploads each image to Cloudinary. - Cloudinary returns secure image URLs. 6. System sends image URLs to the OpenAI Vision model. - System requests: detected vehicle parts + damage type + visibility and confidence. 7. OpenAI returns structured results containing: - Detected car parts - Damage types (replace/repair)

Item	Description
	- Visibility and confidence - Source image index **C. Cost Estimation** 8. System loads internal JSON parts database (organized by year/model/trim/engine). 9. For each detected damaged part: - System searches for the matching part in JSON. - If price not found → use fallback price. 10. System calculates repair/replace labor cost using: - User-provided labor rate - Estimated labor hours per part (from JSON or fallback) 11. System combines AI results + cost data into a structured report object. **D. Storing the Report** 12. System saves the combined results in MongoDB as a pending AI scan report. 13. System returns the results to the frontend, where the user can expand/collapse items in the AIScan UI.
Alternative Flow	A1. AI detects no valid vehicle parts • System displays: “No damage detected — please upload clearer images.” • Users can retry with new images.
Exception Flow	E1. Any system component fails (Cloudinary, OpenAI, file upload, JSON parsing) • System logs the error internally. • System returns: “Analysis Incomplete — An error occurred during the scan.”

Use Case ID: UC-05
Use Case Name: Generate Repair Report

Item	Description
Primary Actor	User
Supporting Actors	None
Goal in Context	Produce a finalized repair report combining AI damage results, parts pricing, and labor cost estimates.
Scope	TrueRepairs Web Application & Backend
Level	System Goal
Preconditions	• User is logged in. • A completed AI Scan exists for the selected vehicle.
Trigger	User clicks **"Generate Report"** in the AI Scan or Report preview popup.
Postconditions	A finalized repair report is created, saved in MongoDB, and displayed in the user's Dashboard.
Main Flow	**A. User Initiates Report Generation** 1. User reviews the AI Scan results and clicks "Generate Report." • System shows a loading indicator. **B. System Builds the Report** 2. System retrieves the pending AI Scan results from MongoDB. 3. System loads internal JSON parts database based on vehicle year, make, model, trim, and engine. 4. For each detected damaged part:

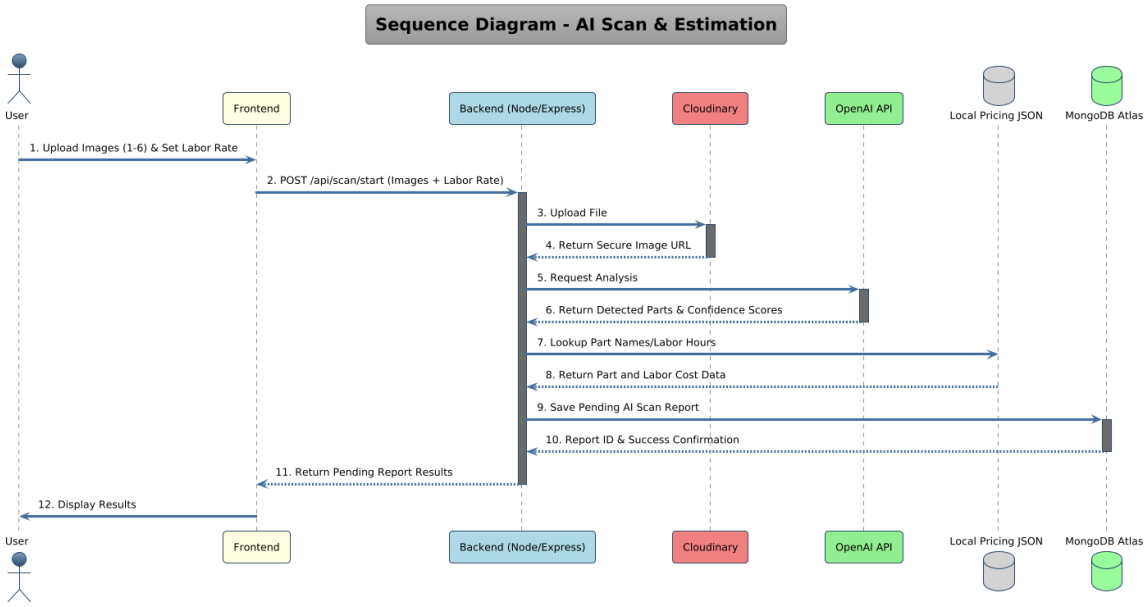
Item	Description
	• System finds the part in the JSON database. • If price not found → apply fallback universal price. • System calculates labor cost using the user's labor rate and estimated hours. 5. System compiles all fields into a structured report payload, including: • Detected parts • Replace/repair recommendation • Accuracy values • Part cost • Labor cost • Total estimated cost • Image references • User & vehicle metadata • Timestamp **C. Save & Provide Output** 6. System saves the finalized report document into MongoDB under the user's account. 7. System updates the Dashboard, so the new report appears in the Report History list. 8. System displays success message: • "Report Generated Successfully."
Alternative Flow	A1. User Attempts to Generate a Report Twice • System detects an existing finalized report for the same AI Scan. • System displays: "A finalized report already exists. Loading existing report…" • Existing reports are shown instead of creating a duplicate.
Exception Flow	E1. Database Write Failure • System retries once. • If still failing → logs the error and displays: "Report could not be saved. Please try again later."

Use Case ID: UC-06
Use Case Name: View or Delete Report

Item	Description
Primary Actor	User
Goal in Context	Allow users to view, download, or delete their reports.
Scope	TrueRepairs Web Application
Level	User Goal
Preconditions	At least one report exists for the user.
Trigger	The user clicks the “Manuel Reports” button or menu option from the Dashboard.
Postconditions	The report is viewed or removed from database.
Main Flow	1. User opens Reports page. 2. System displays available reports. 3. User selects “View” or “Delete.” 4. System executes request and updates Dashboard.
Alternative Flow	A1. User cancels deletion → System returns to Dashboard.
Exception Flow	E1. Network failure → System shows “Action Failed.”

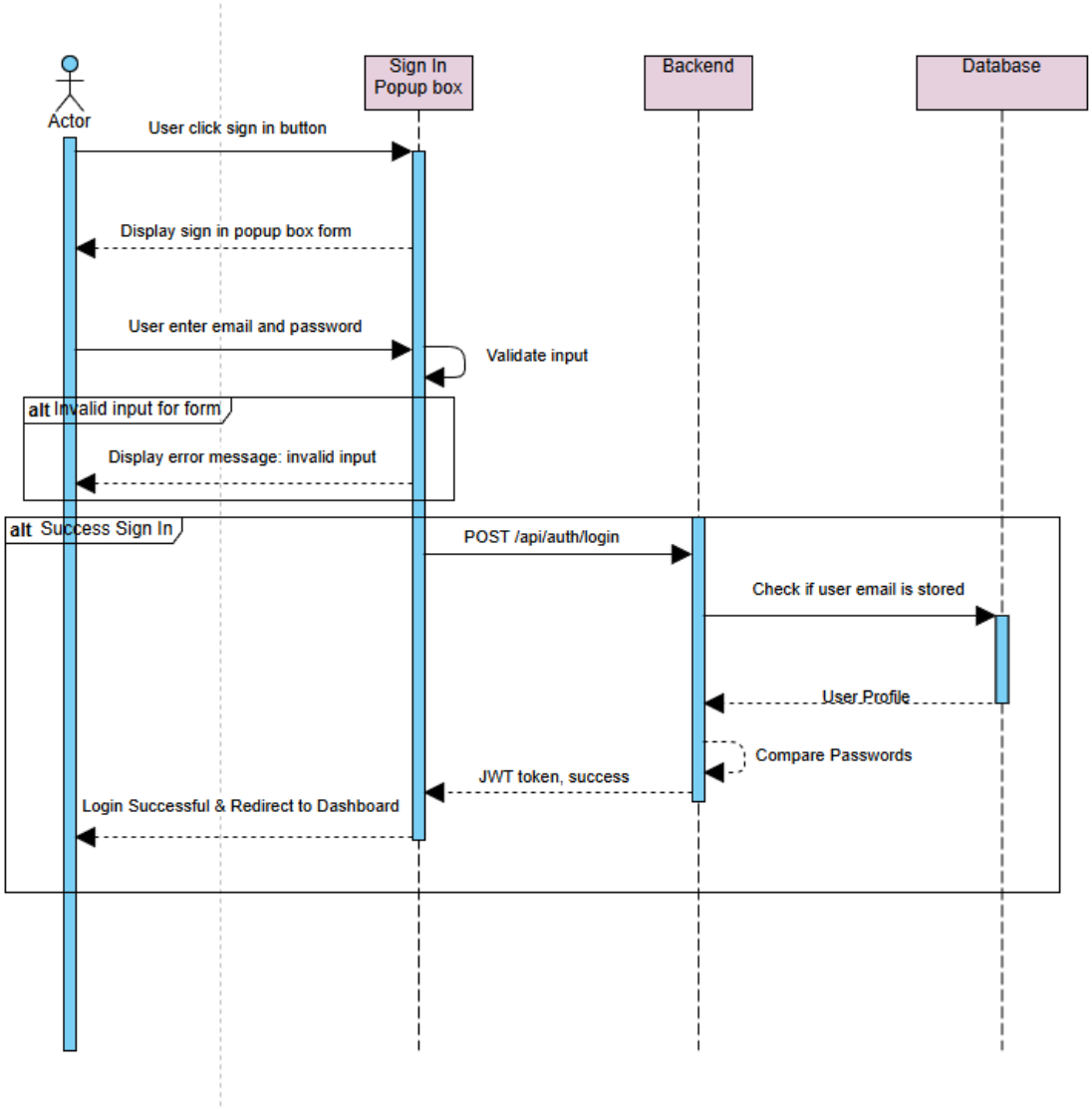
4.2 Sequence Diagram

4.2.1 AI Scan Flow

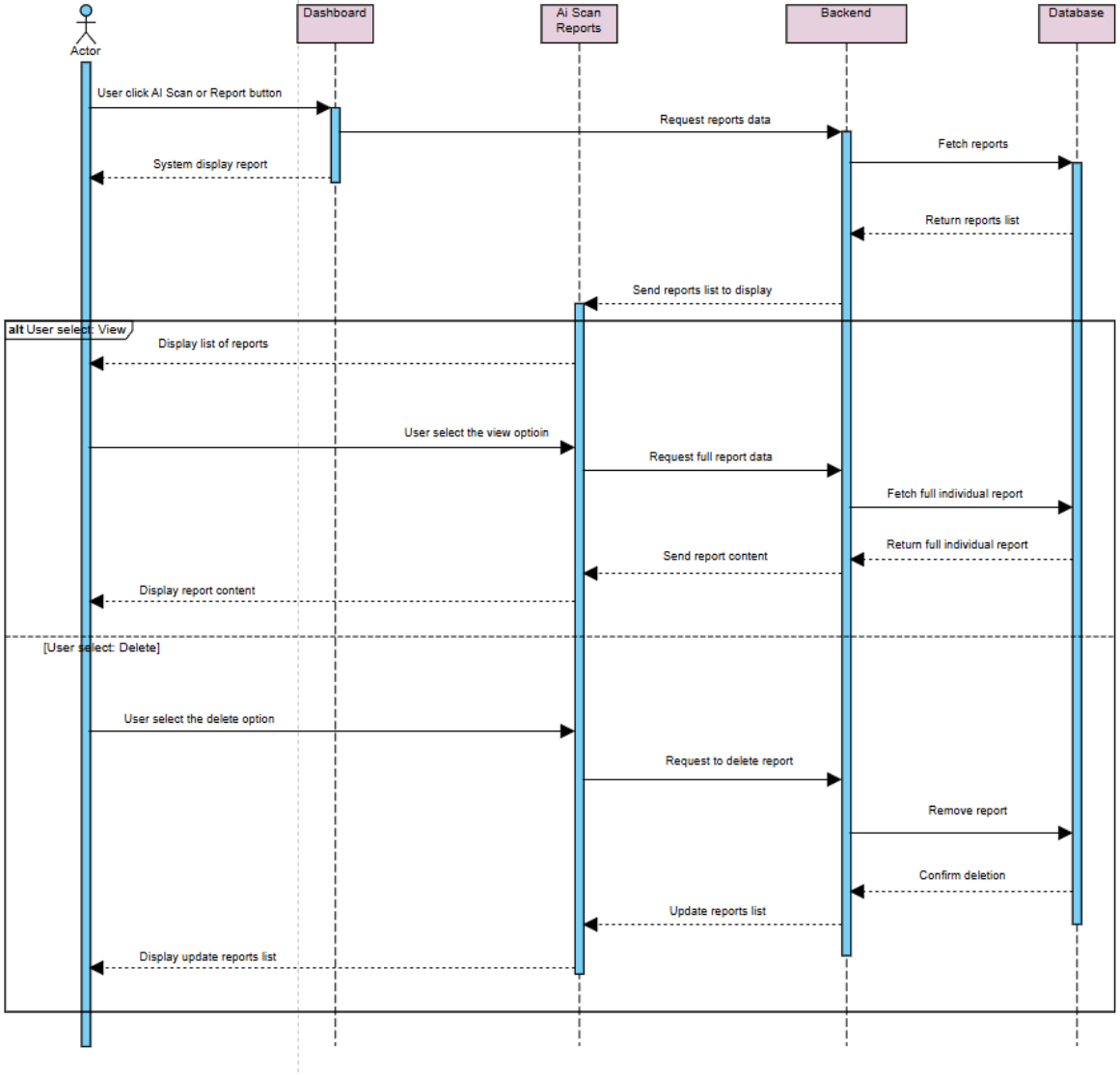


Displays the flow of the AI Scan operation from user upload to report delivery, including Cloudinary, OpenAI, JSON Parts Data, and MongoDB.

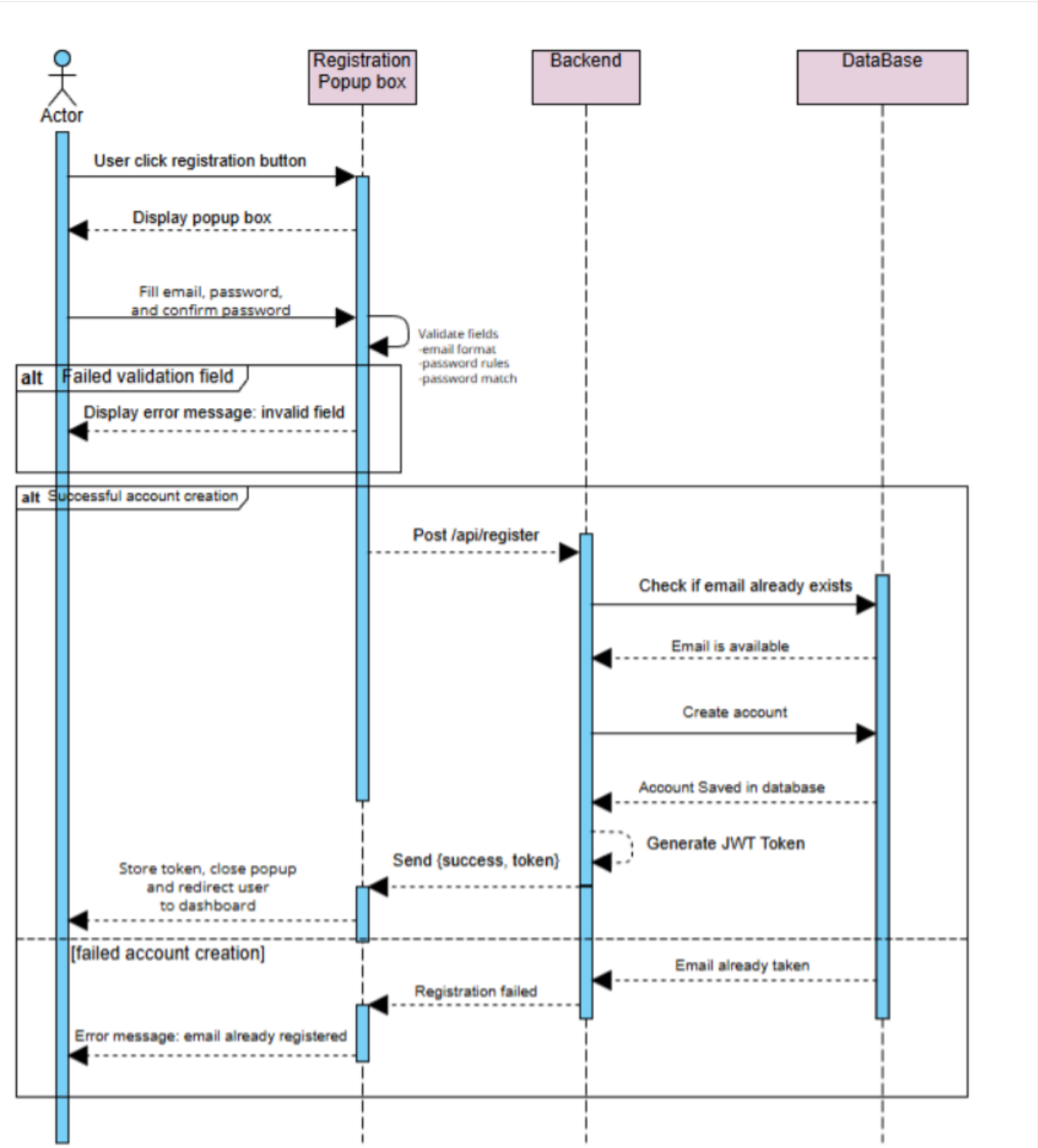
4.2.2 User Login



4.2.3 AI View Final Report

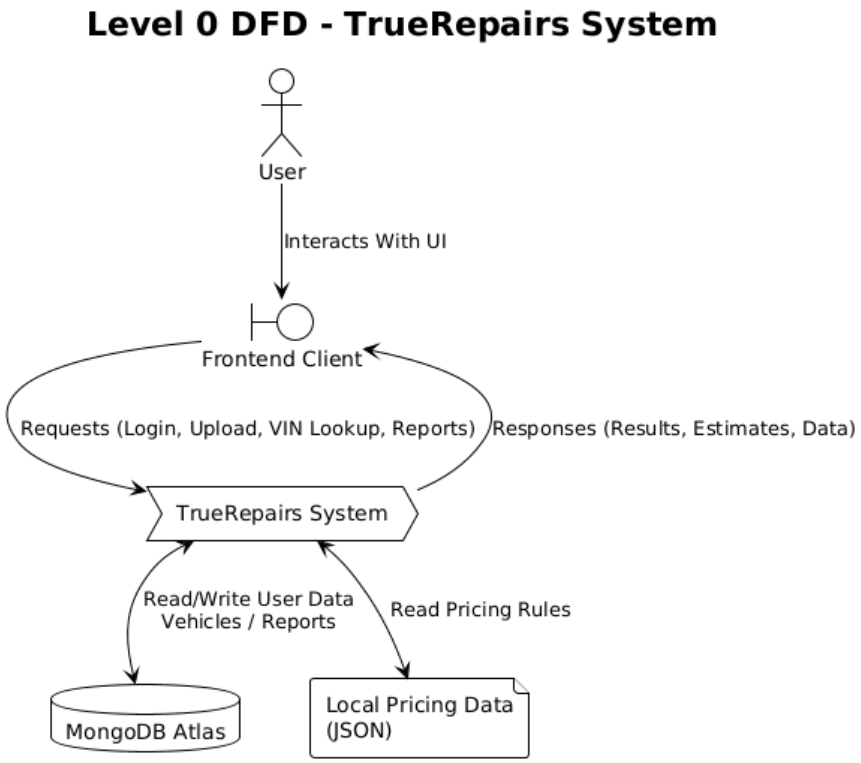


4.2.4 User Registration



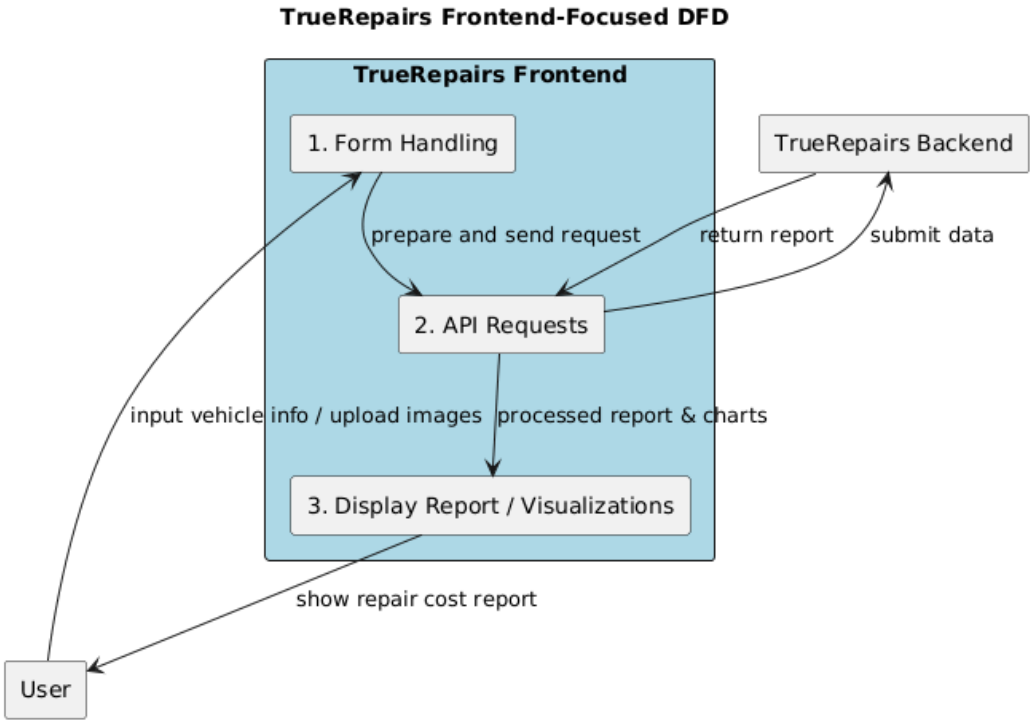
4.3 Data Flow Diagrams

4.3.1 TrueRepairs Data Flow Diagram - Level 0

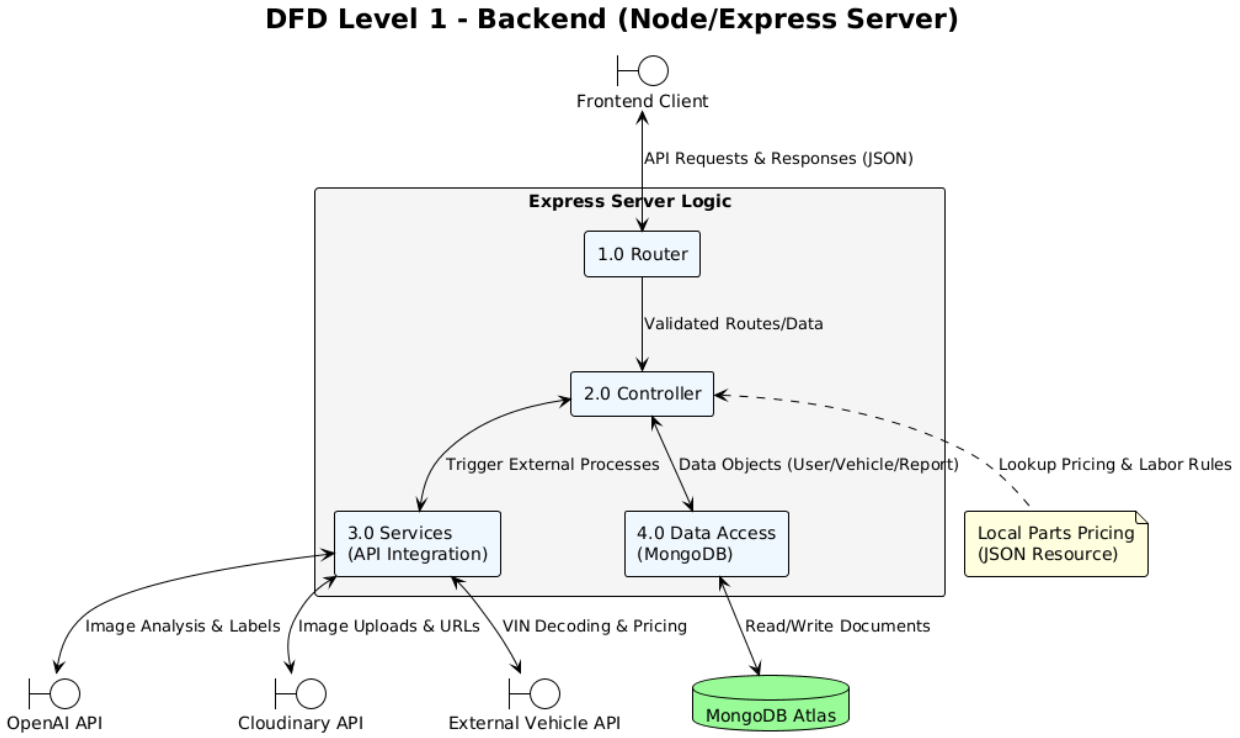


Illustrates the path of data between users, the TrueRepairs app, and external systems.

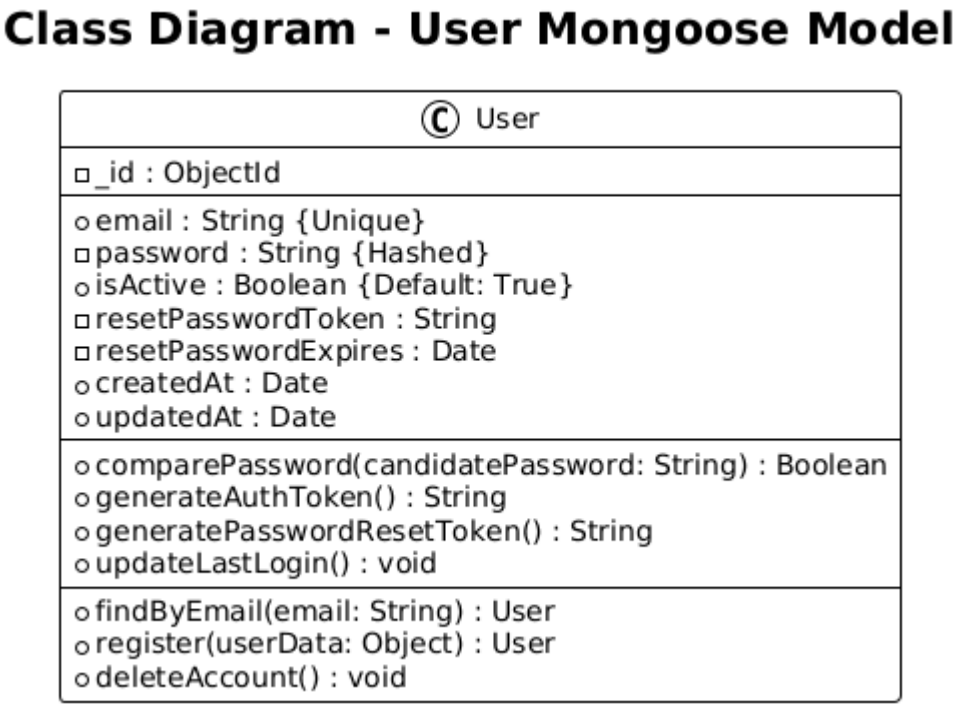
4.3.2 TrueRepairs Data Flow Diagram - Level 1 (Frontend)



4.3.3 TrueRepairs Data Flow Diagram - Level 1 (Backend)



4.4 Class Diagram



4.5 API Endpoints Overview

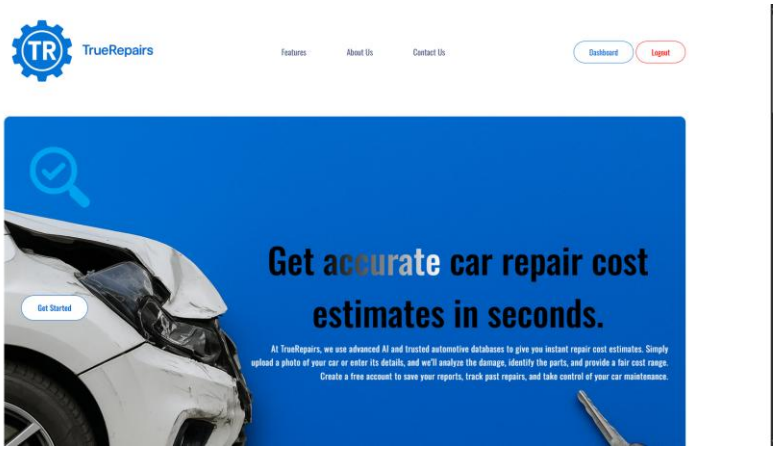
Endpoint	Method	Description
/auth/register	POST	Create new user account
/auth/login	POST	Authenticate and issue JWT
/vehicles	POST	Add vehicle profile
/ai/scan	POST	Process image and generate report
/reports	GET	Retrieve all reports
/reports/:id	DELETE	Delete report by ID

4.6 User Interface Design

Overview

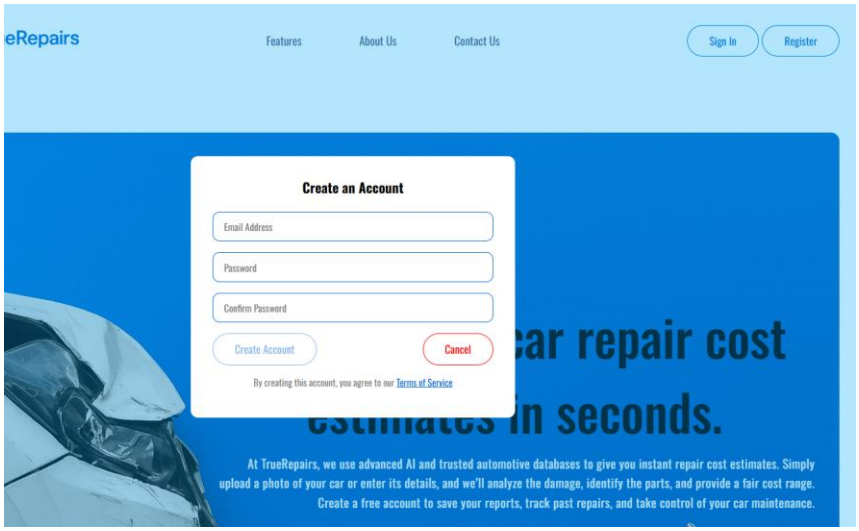
The TrueRepairs interface is designed for clarity, consistency, and ease of navigation.
Each page uses a responsive blue-and-white theme with rounded components, clean typography, and a three-click rule for all major actions.

Figure 4.6.1 – Homepage



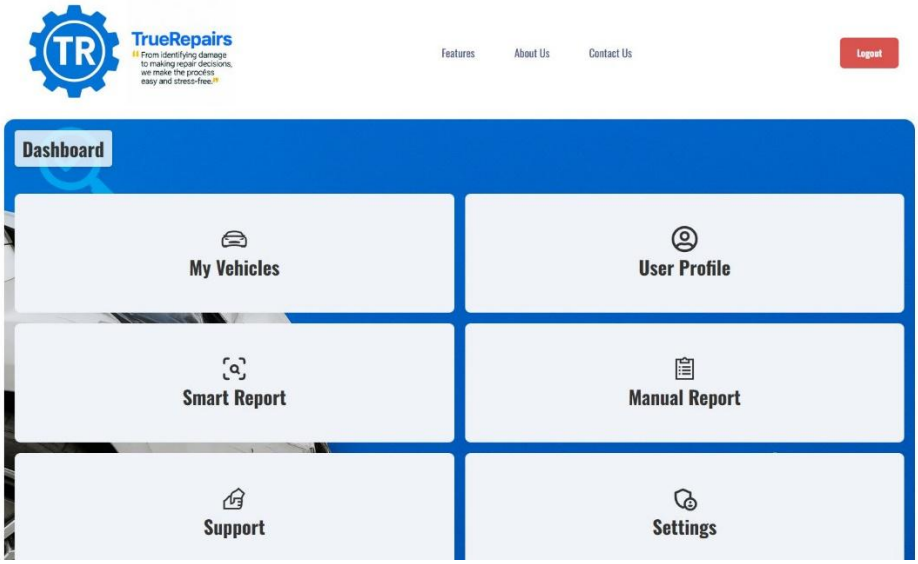
The homepage introduces the platform, highlights core features (AI Scan, Vehicle Lookup, Repair Reports), and provides quick access to login and registration.
The design emphasizes simplicity and high contrast for readability.

Figure 4.6.2 – Registration / Login Page



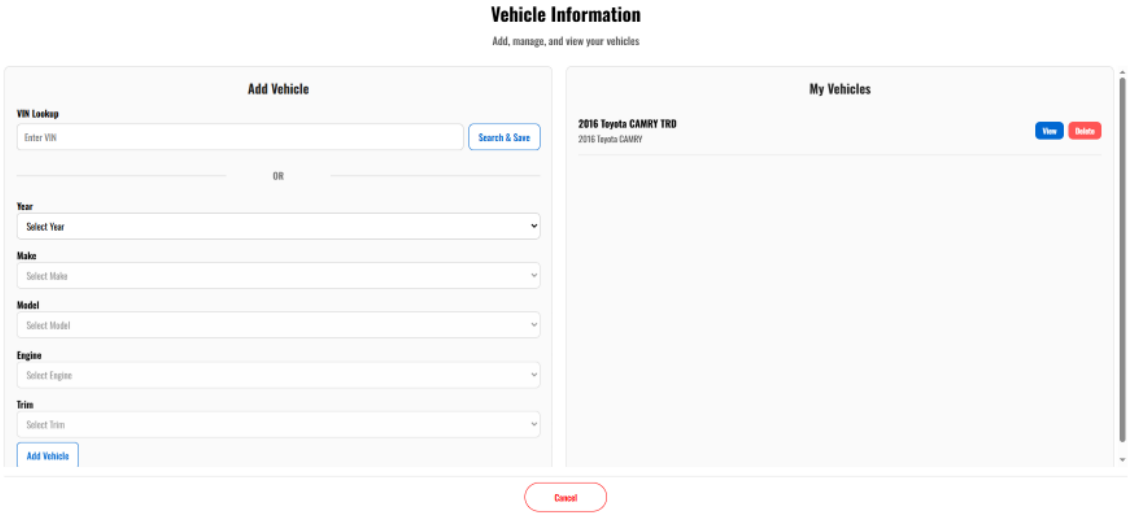
Displays input fields for email, username, and password with validation feedback.
Includes links for “Forgot Password” and “Create Account,” following the same color and typography guidelines.

Figure 4.6.3 – User Dashboard



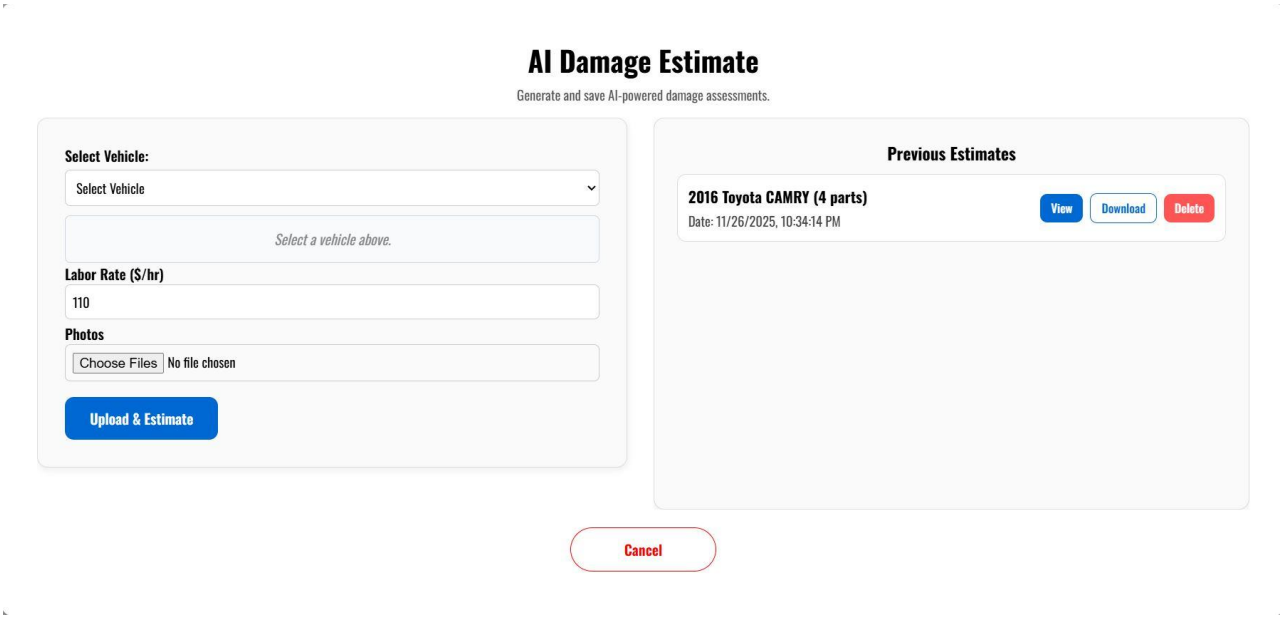
After authentication, the user lands on the Dashboard.
It displays vehicle summaries, recent repair reports, and buttons to start new AI Scans or manage vehicles.

Figure 4.6.4 – Vehicle Information Dashboard



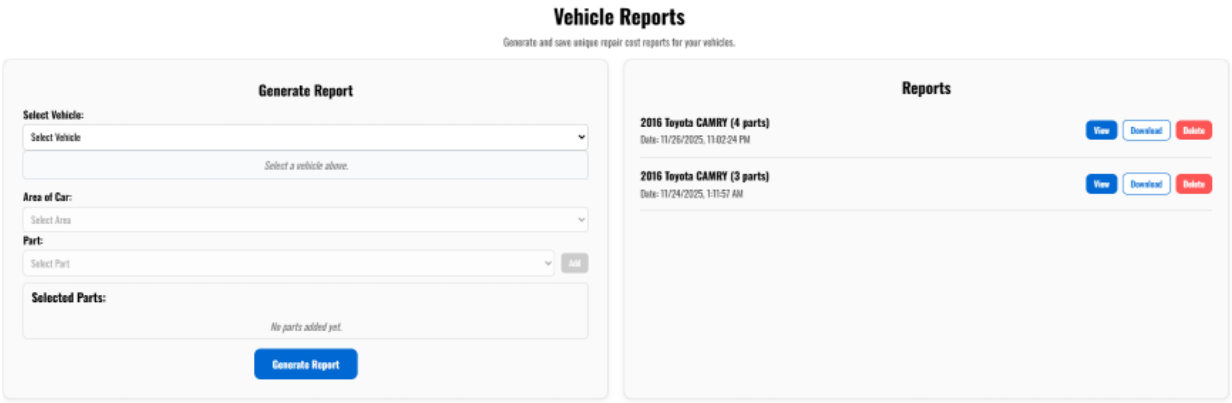
After authentication and selection of the “My Vehicles” section from the main dashboard, the user is presented with the Vehicle Information Dashboard View. This interface provides an organized summary of all vehicles associated with the user’s account.

Figure 4.6.4 – AI Scan Page



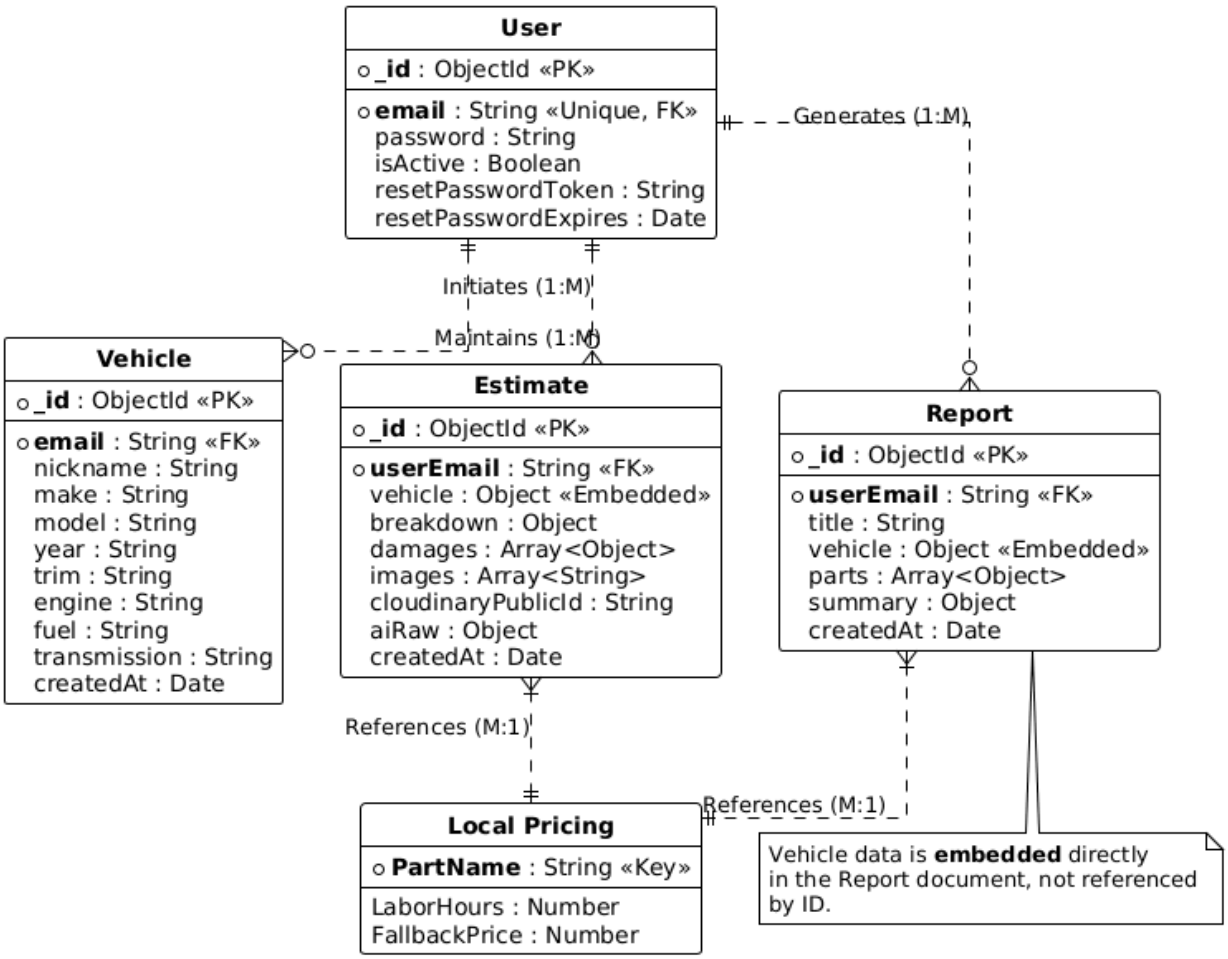
The AI Scan interface allows users to upload vehicle images and trigger analysis.
Progress indicators and result sections display real-time feedback as the backend processes OpenAI and JSON Parts data data.

Figure 4.6.5 – Repair Report Page



Displays the finalized report containing detected damage areas, estimated parts and labor costs, and total DIY costs.
The page allows exporting or deleting a report directly from the dashboard.

4.7 Database Design Diagram



Entities:

- **User** - email, passwordHash, isActive
- **Vehicle** – email, nickname, vin, make, model, year, trim, engine, fuel, transmission
- **Report** – email, vehicle, title, parts[], createdAt
- **Estimate** – email, vehicle, images, breakdown, damages, createdAt

Relationships:

- One User → Many Vehicles

- One Vehicle → Many Repair Report

5.0 References

- OpenAI Vision API Documentation
- Cloudinary Upload API Documentation
- MongoDB Atlas Documentation
- Node.js & Express Official Documentation
- React & Vite Official Documentation
- IEEE 830-1998 Software Design Specification Standard